

**SPESIFIKASI KEBUTUHAN PERANGKAT LUNAK (SKPL)
APLIKASI UMKM PENJUALAN SAYURAN SEGAR
KELOMPOK - 3**



Anggota kelompok:

20231310046	M. Irvan Alfiansyah
20231310047	M. Nur Yanfa
20231310061	Ilham Ramdan

**PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA**

*Jln. Terusan Halimun No. 37 (Pelajar Pejuang 45) Lingkar Selatan Kec. Lengkong
kota Bandung Jawa Barat 40263*

	<i>Program Studi Teknik Informatika Fakultas Ilmu Komputer Dan Sistem Informasi Universitas Kebangsaan Republik Indonesia</i>	Nomor dokumen		Halaman
		SKPL-UMKM-003		1/10
		Revisi		Maret 2026

DAFTAR PERUBAHAN

Revisi	Deskripsi
A	Versi pertama — dokumen SKPL Aplikasi UMKM Sayuran Segar
B	
C	
D	
E	
F	
G	

Index	A	B	C	D	E	F	G
TGL							
Ditulis Oleh							
Diperiksa Oleh							
Disetujui Oleh							

DAFTAR HALAMAN PERUBAHAN

HALAMAN	REVISI	HALAMAN	REVISI

DAFTAR ISI

Contents

I. Pendahuluan	6
1.1 Tujuan Penulisan Dokumen.....	6
1.2 Lingkup Masalah.....	6
1.3 Definisi dan Istilah	7
1.4 Aturan Penamaan dan Penomoran	8
1.5 Referensi.....	9
1.6 Ikhtisar Dokumen.....	9
1.7 Kebutuhan Non-Fungsional	9
II. DESKRIPSI PERANCANGAN GLOBAL	10
2.1 Rancangan Lingkungan Implementasi	11
2.2 Model Proses Berbasis Objek	11
2.2.1 Diagram Use Case	12
2.2.2 diagram sequence.....	14
2.2.3 Diagram Implementasi.....	18
2.3 Model Data	20
2.3.1 Definisi Domain/Type.....	20
2.3.2 Data Model Logika.....	21
2.3.3 Data Model Fisik.....	Error! Bookmark not defined.
2.3.4 daftar tabel aplikasi	23
III. DESKRIPSI PERANCANGAN RINCI	25
3.1 Deskripsi Rinci Tabel	25
3.1.1 Tabel User	25
3.1.2 Tabel Produk.....	26
3.1.3 Tabel Keranjang	26
3.1.4 Tabel Pesanan.....	27
3.1.5 Tabel Pembayaran.....	Error! Bookmark not defined.
3.1.6 Tabel Pengiriman	Error! Bookmark not defined.
3.2 Deskripsi Proses Secara Rinci	Error! Bookmark not defined.
3.2.1 Spesifikasi Proses: Autentikasi Pengguna (Login API).....	29
3.2.2 Spesifikasi Proses: Manajemen Katalog & Kepemilikan (Khusus Supplier)	32
3.2.3 Spesifikasi Proses: Checkout dan Penguncian Stok ACID (Khusus Pembeli)	33

3.2.4 Spesifikasi Proses: Rekonsiliasi Pembayaran (Webhook Midtrans)	37
3.2.5 Spesifikasi Proses: Manajemen Logistik (Khusus Kurir)	38
3.3 Dekomposisi Fisik Modul	40

I. PENDAHULUAN

1.1 Tujuan Penulisan Dokumen

Tujuan utama dari penyusunan dokumen Spesifikasi Kebutuhan Perangkat Lunak (SKPL) ini adalah untuk mendefinisikan, menguraikan, serta mendokumentasikan secara komprehensif dan absolut seluruh kebutuhan fungsional dan non-fungsional dari sistem **Aplikasi E-Commerce UMKM Sayuran Segar Berbasis RESTful API**. Dokumen ini tidak sekadar menjadi formalitas administratif, melainkan diposisikan sebagai landasan rekayasa perangkat lunak strategis yang berorientasi kuat pada pengujian kualitas (*Software Quality Assurance / SQA*) dan ketahanan keamanan sistem informasi .

Dalam siklus pengembangan perangkat lunak (*Software Development Life Cycle / SDLC*), dokumen ini menduduki fungsi sentral sebagai kontrak teknis mutlak antar-pemangku kepentingan. Bagi tim pengembang (*software engineers*), dokumen ini bertindak sebagai cetak biru (*blueprint*) arsitektur dalam mengimplementasikan kode sumber *backend* menggunakan pola *Clean Architecture* di bahasa pemrograman Golang, serta memastikan bahwa titik akhir (*endpoints*) API yang dibangun dapat diintegrasikan secara presisi dengan *frontend* berbasis React.js. Bagi analis penjamin mutu (*Quality Assurance / QA Engineers*), dokumen ini merupakan panduan utama dalam merancang matriks pengujian ekstrem, yang mencakup pengujian kotak putih (*White-Box Testing*) pada logika internal, pengujian kotak hitam (*Black-Box Testing*) pada fungsionalitas antarmuka, pengujian penetrasi keamanan lintas-aktor (*Security/IDOR Testing*), hingga pengujian beban komputasi maksimal (*Stress/Load Testing*).

1.2 Lingkup Masalah

Sistem yang dikembangkan, yakni Aplikasi E-Commerce UMKM Sayuran Segar API, merupakan sebuah ekosistem perangkat lunak terdistribusi yang memfasilitasi orkestrasi rantai pasok dan digitalisasi proses jual beli komoditas pertanian dari hulu (petani/pengepul) hingga ke hilir (konsumen akhir). Menjawab tantangan proses bisnis di dunia nyata yang menuntut performa komputasi tinggi tanpa mengorbankan pengalaman pengguna, arsitektur sistem ini dibangun dengan pendekatan *Decoupled Architecture*. Pendekatan ini memisahkan secara total antara antarmuka pengguna di sisi klien (*Client-Side Rendering* dengan React.js) dan peladen pemroses logika murni di sisi belakang (*Backend API* dengan Golang).

Ruang lingkup penyelesaian masalah pada sistem ini merepresentasikan digitalisasi proses bisnis UMKM yang utuh dan menyeluruh. Pembatasan sistem (*system boundary*) difokuskan pada pemrosesan lima pilar modul komputasi utama yang sarat akan titik pengujian SQA, yaitu:

1. **Modul Autentikasi dan Multi-Otorisasi Hierarkis:** Menangani manajemen akses masuk terpusat untuk empat entitas pengguna yang memiliki batasan wewenang berbeda (Admin, Pembeli, Supplier, dan Kurir). Modul ini mengamankan profil pengguna dengan sandi terenkripsi kriptografi dan menerbitkan sesi akses berupa token digital berbatas waktu (*stateless session*).
2. **Modul Manajemen Katalog dan Kepemilikan (Supplier & Admin):** Memfasilitasi aktor Supplier (petani/pegepul) untuk mendaftarkan dan memutakhirkan ketersediaan stok panen mereka secara mandiri ke dalam etalase platform. Ruang lingkup modul ini diperketat dengan perlindungan kepemilikan data absolut (*Ownership Protection*), di mana peladen secara proaktif mencegah celah keamanan sehingga seorang Supplier dipastikan tidak akan pernah dapat memodifikasi atau menghapus produk sayuran yang diunggah oleh Supplier lain.
3. **Modul Transaksi dan Pemesanan Berstandar ACID:** Mengakomodasi dan memvalidasi proses pembeli dalam menyusun keranjang belanja hingga tahap penyelesaian pesanan (*checkout*). Ini adalah inti dari pengujian kualitas sistem, di mana modul menerapkan prinsip transaksi basis data terisolasi untuk menjamin bahwa algoritma pemotongan stok tidak akan pernah menghasilkan angka minus (*overselling*) meskipun produk yang sama dibeli oleh ratusan pengguna secara serentak dalam hitungan milidetik.
4. **Modul Integrasi Pembayaran Asinkronus (Payment Gateway):** Menangani rekonsiliasi dan sinkronisasi status pelunasan pesanan secara otomatis tanpa intervensi manusia. Modul ini beroperasi melalui validasi tanda tangan kriptografi (*Signature Key Validation*) dari *Webhook* sistem pihak ketiga (Midtrans) guna mencegah manipulasi status pembayaran oleh peretas.
5. **Modul Pelacakan Logistik Fisik (Kurir):** Memfasilitasi armada pengiriman atau Kurir untuk memutakhirkan titik status perjalanan fisik barang (seperti 'Dikirim' atau 'Diterima'). Status ini dipancarkan kembali melalui API sehingga Pembeli dapat melacak pesanan mereka secara transparan.

1.3 Definisi dan Istilah

Guna menghindari ambiguitas semantik dan menyamakan persepsi komprehensif dari berbagai pemangku kepentingan, berikut adalah definisi, singkatan, dan terminologi teknis rekayasa perangkat lunak yang digunakan di sepanjang dokumen ini :

- **ACID (Atomicity, Consistency, Isolation, Durability):** Seperangkat jaminan properti transaksi pada basis data relasional. Properti ini memastikan bahwa transaksi multi-langkah (seperti kalkulasi harga dan pemotongan stok saat *checkout*) akan dieksekusi secara tuntas seluruhnya atau dibatalkan sepenuhnya (*rollback*) jika terjadi anomali, guna menghindari korupsi data.

- **API (Application Programming Interface):** Kontrak aturan komunikasi data dua arah berbasis format JSON yang menghubungkan aplikasi *frontend* (etalase visual) dengan peladen *backend* (otak komputasi bisnis).
- **Bcrypt:** Algoritma fungsi *hash* kriptografi yang secara sengaja dirancang untuk berjalan lambat secara komputasional (*cost-based*). Algoritma ini digunakan untuk menyandikan kata sandi pengguna agar kebal terhadap serangan peretasan tipe *brute-force* maupun pencocokan *rainbow table*.
- **Clean Architecture:** Paradigma perancangan arsitektur perangkat lunak yang memisahkan kode sumber *backend* menjadi lapisan-lapisan independen (meliputi *Delivery*, *Usecase*, *Repository*, dan *Domain*). Tujuannya adalah memastikan logika bisnis inti tidak bergantung pada kerangka kerja web maupun jenis basis data eksternal.
- **IDOR (Insecure Direct Object Reference):** Kerentanan keamanan siber tingkat tinggi yang terjadi ketika aplikasi memberikan akses langsung ke basis data hanya berdasarkan manipulasi input parameter pengguna. Sistem ini diuji secara ketat untuk mendeteksi dan mencegah IDOR.
- **JWT (JSON Web Token):** Standar terbuka (RFC 7519) untuk merepresentasikan klaim informasi secara aman antara dua pihak. Dalam ekosistem ini, JWT digunakan untuk mengamankan sesi identitas dan *Role* pengguna tanpa membebani memori peladen (*stateless authentication*).
- **Middleware:** Perangkat lunak perantara yang bertugas mencegah (*intercept*) permintaan HTTP sebelum mencapai pusat logika aplikasi. Digunakan secara luas dalam sistem ini untuk fungsi validasi token, pencatatan log (*logger*), dan pencegahan matinya peladen (*panic recovery*).
- **MVCC (Multi-Version Concurrency Control):** Fitur pada basis data PostgreSQL yang memungkinkan sistem menangani proses pembacaan data katalog secara bersamaan tanpa memblokir proses penulisan data transaksi, sangat krusial untuk mempertahankan kecepatan peladen di bawah tekanan lalu lintas tinggi.
- **RPS (Requests Per Second):** Metrik parameter analitik utama dalam pengujian beban (*load testing*) yang mendokumentasikan seberapa banyak lalu lintas permintaan masuk yang dapat diproses dan dijawab oleh peladen dalam durasi satu detik.
- **Webhook:** Konsep integrasi *HTTP Push API* di mana peladen Midtrans berinisiatif mengirimkan muatan data secara otomatis (tanpa diminta) ke peladen UMKM secara *real-time* setiap kali pelanggan menyelesaikan pembayaran di layanan perbankan.

1.4 Aturan Penamaan dan Penomoran

Pengembangan sistem dan penyusunan spesifikasi teknis di dalam dokumen SKPL ini tunduk pada konvensi penamaan baku guna menjaga konsistensi, tingkat keterbacaan kode (*readability*), dan kemudahan pelacakan matriks pengujian :

1. **Konvensi Kode Backend Golang:** Seluruh penamaan berkas fisik kode sumber diwajibkan menggunakan format huruf kecil yang dipisah garis bawah atau *snake_case*

(sebagai contoh: *category_repository.go*). Sementara itu, penamaan struktur entitas (*Struct*) dan antarmuka (*Interface*) menggunakan format *PascalCase* (sebagai contoh: *ProductUseCase*).

2. **Konvensi Entitas Basis Data:** Identitas pada tingkat basis data relasional mematuhi konvensi bahasa Indonesia yang diselaraskan dengan glosarium proses bisnis nyata. Nama tabel menggunakan huruf kecil dengan format *snake_case* (seperti *users*, *produk*, *pengiriman*), begitu pula dengan nama kolom atributnya (seperti *id_pesanan*, *nama_produk*).
3. **Konvensi Rute Titik Akhir API:** Penamaan jalur alamat URL untuk akses API disusun menggunakan kaidah kata benda jamak berbasis *kebab-case* dan menyertakan indikator versi untuk kemudahan pemeliharaan berjangka panjang (sebagai contoh: *POST /api/v1/shopping-carts*).
4. **Konvensi Penomoran Pengujian SQA:** Seluruh skenario pengujian kualitas dalam dokumen ini dikodifikasi menggunakan format yang memadukan singkatan modul dan nomor urut. Format baku yang digunakan adalah *[KODE_MODUL]-[NOMOR_URUT]* (sebagai contoh: *AUTH-01* merujuk pada pengujian modul autentikasi pertama, dan *CHK-02* merujuk pada pengujian modul *checkout* kedua).

1.5 Referensi

Penyusunan dokumen Spesifikasi Kebutuhan Perangkat Lunak ini dilandaskan pada perpaduan literatur standar akademis formal dan dokumentasi teknis dari perintis industri perangkat lunak global. Referensi utama yang menjadi rujukan meliputi:

1. Panduan Institusional dan Template Dokumen SKPL Model Berbasis Objek (OO) Standar Perguruan Tinggi.
2. Standar Internasional Rekayasa Perangkat Lunak IEEE 830-1998 mengenai *Recommended Practice for Software Requirements Specifications*.
3. Konsorsium Standar Keamanan Aplikasi Web Global dari OWASP (*Open Web Application Security Project*) Top 10 Security Risks.
4. Dokumentasi Resmi Pustaka Inti Bahasa Pemrograman Go (Golang), Kerangka Kerja Jaringan Gin, dan Modul Relasi Objek GORM.
5. Dokumentasi Resmi Antarmuka *Frontend* React.js dan Manajemen Rute *Single Page Application*.
6. Dokumentasi Teknis Integrasi *Payment Gateway* dan Standar Kriptografi API Midtrans.

1.6 Ikhtisar Dokumen

Sistematika pemaparan di dalam dokumen SKPL ini dirancang secara berjenjang dari abstraksi konseptual menuju spesifikasi komputasi teknis, dan dibagi menjadi tiga bab utama .

- **Bab I: Pendahuluan**, merupakan fondasi dokumen yang memaparkan latar belakang rasionalisasi sistem, tujuan penyusunan, definisi ruang lingkup proses bisnis utuh (hulu ke hilir), penjabaran istilah teknis, aturan penamaan standar baku, serta daftar referensi literatur.
- **Bab II: Deskripsi Perancangan Global**, merupakan pembedahan arsitektur sistem makro. Bab ini menguraikan lingkungan implementasi perangkat lunak, pemodelan proses berorientasi objek lintas-aktor yang divisualisasikan melalui diagram *Use Case*, *Sequence*, *Komponen*, dan *Deployment*. Bab ini juga meletakkan rancangan arsitektur pemodelan data dari entitas logis menuju struktur tabel fisik.
- **Bab III: Deskripsi Perancangan Rinci**, merupakan inti komputasional dari dokumen. Bab ini merinci parameter fisik setiap kolom basis data, menjabarkan secara eksak spesifikasi kontrak algoritma dan antarmuka API (menggantikan spesifikasi layar visual konvensional), dan ditutup dengan pemaparan matriks skenario pengujian kualitas sistem (SQA) sebagai parameter keberhasilan produk akhir.

1.7 Kebutuhan Non-Fungsional

Selain kebutuhan fungsional yang telah dijabarkan, sistem wajib memenuhi kebutuhan non-fungsional berikut:

1. Performa: Waktu respons API untuk operasi standar tidak melebihi 500ms pada kondisi normal (< 100 concurrent users).
2. Skalabilitas: Sistem harus menangani minimal 500 Requests Per Second (RPS) tanpa degradasi, dapat di-scale horizontal via Docker.
3. Ketersediaan: Server backend Golang harus memiliki uptime minimum 99.5% per bulan.
4. Keamanan: Sistem wajib lolos pengujian penetrasi untuk: SQL Injection (GORM parameterized query), Broken Authentication (JWT), IDOR (validasi kepemilikan), Brute-force (bcrypt cost 14).
5. Konsistensi Data: Tidak boleh terjadi overselling (stok < 0). Seluruh operasi checkout menggunakan transaksi ACID dengan row-level locking PostgreSQL.
6. Kemudahan Perawatan: Arsitektur Clean Architecture (Delivery -> Usecase -> Repository -> Domain).
7. Kompatibilitas: API dapat dikonsumsi oleh klien React.js dan mengikuti standar RESTful JSON API.

II. DESKRIPSI PERANCANGAN GLOBAL

2.1 Rancangan Lingkungan Implementasi

Sistem Aplikasi E-Commerce UMKM Sayuran Segar ini dirancang secara khusus untuk beroperasi pada lingkungan implementasi terdistribusi modern yang menerapkan prinsip pemisahan secara mutlak antara antarmuka pengguna di sisi klien (*frontend*) dan komputasi logika bisnis di sisi peladen (*backend*). Pemisahan arsitektur (*decoupled architecture*) ini merupakan keputusan strategis untuk mempermudah audit keamanan, mempercepat pelokalan *bug* (cacat perangkat lunak), serta memungkinkan tim jaminan mutu (QA) melakukan pengujian beban (*load testing*) pada peladen secara independen tanpa dipengaruhi oleh performa *rendering* visual di peramban pengguna.

Pada lapisan antarmuka visual, sistem mengasumsikan keberadaan aplikasi *Single Page Application* (SPA) berbasis pustaka React.js yang merepresentasikan etalase *marketplace* fungsional kelas industri. Antarmuka ini bertugas menangani interaksi dom (*Document Object Model*) dan navigasi pengguna secara dinamis, untuk kemudian berkomunikasi dengan pusat data melalui permintaan HTTP asinkronus.

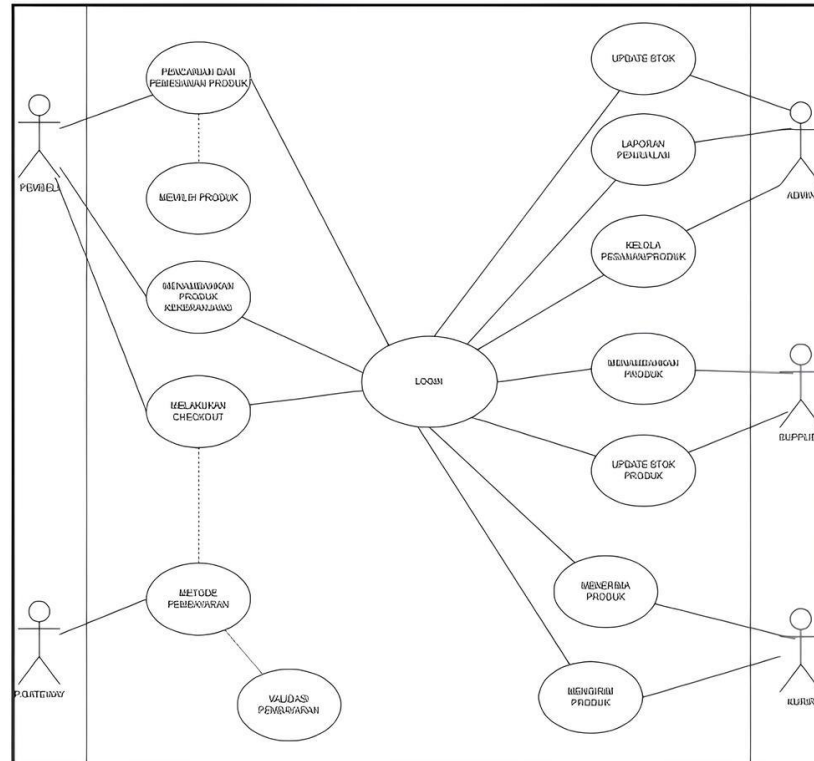
Beralih pada sisi peladen, komputasi logika bisnis sepenuhnya digerakkan oleh bahasa pemrograman Go (Golang) versi 1.20 atau yang lebih baru. Golang dipilih sebagai tulang punggung arsitektur karena keunggulannya dalam mengelola ribuan eksekusi proses secara paralel (*concurrency*) melalui *Goroutine* dengan konsumsi alokasi memori yang sangat efisien. Ekosistem peladen ini ditopang oleh kerangka kerja jaringan *Gin Web Framework* untuk memfasilitasi titik rute API yang sangat cepat, serta pustaka *Object-Relational Mapping* (GORM) untuk menerjemahkan struktur objek ke dalam relasi basis data sekaligus melakukan sanitasi kueri otomatis guna menangkal ancaman *SQL Injection*. Sementara itu, pangkalan data dikelola menggunakan sistem manajemen basis data relasional PostgreSQL versi 15. PostgreSQL diimplementasikan sebagai fondasi penyimpanan absolut karena kepatuhannya terhadap standar transaksi ACID (*Atomicity, Consistency, Isolation, Durability*) dan kapabilitas *Multi-Version Concurrency Control* (MVCC) yang sangat krusial dalam ekosistem *e-commerce*.

2.2 Model Proses Berbasis Objek

Pemodelan proses perangkat lunak pada sistem ini mengadaptasi paradigma berorientasi objek yang diselaraskan secara ketat dengan prinsip *Clean Architecture*. Pendekatan ini memastikan bahwa setiap entitas bisnis dan alur kerja di dalam ekosistem rantai pasok UMKM direpresentasikan sebagai objek mandiri yang memiliki tanggung jawab tunggal (*Single Responsibility Principle*).

2.2.1 Diagram Use Case

Sistem mengakomodasi dan membatasi interaksi fungsional lintas-sektoral melalui lima aktor utama yang masing-masing dikurung dalam ruang lingkup kewenangan terisolasi .



- Aktor Pembeli: Pengguna akhir di hilir rantai pasok yang memiliki otoritas untuk menavigasi katalog sayuran, memanipulasi keranjang belanja, mengeksekusi komitmen pemesanan komersial (*checkout*), serta melacak status perjalanan barang yang diantar secara waktu nyata.
- Aktor Admin: Entitas pengawas ekosistem yang dilindungi oleh perlindungan perantara (*Role-Based Access Control*). Admin memiliki hak istimewa (*privilege*) untuk memonitor lalu lintas transaksi finansial, mengelola referensi kategori induk, dan melakukan intervensi moderasi terhadap data pengguna maupun katalog apabila terdeteksi pelanggaran.

- Aktor Supplier (Petani/Pengepul): Produsen komoditas di hulu rantai pasok yang berwenang untuk mendaftarkan hasil panen mereka secara mandiri ke dalam platform. Sistem secara komputasional mengunci kewenangan mereka sehingga seorang Supplier hanya mampu mengubah harga jual dan kuantitas stok untuk produk yang mereka tautkan dengan identitas akun mereka sendiri (*Ownership Protection*).
- Aktor Kurir: Entitas logistik operasional yang bertugas memindahkan wujud fisik komoditas. Melalui antarmuka API, Kurir berwenang memperbarui titik pelacakan barang (seperti 'Dikemas', 'Diantar', hingga 'Diterima oleh Pembeli') khusus untuk manifes pesanan yang telah ditugaskan kepadanya.
- Aktor Payment Gateway (Midtrans): Satu-satunya aktor berupa mesin eksternal yang diizinkan untuk mengintervensi status pesanan aplikasi. Aktor ini berinteraksi secara asinkronus dengan menembakkan muatan data berformat JSON (*Webhook*) ke rute khusus di peladen Golang guna menginformasikan perubahan status pelunasan transaksi.

Berikut narasi detail lima use case utama sistem.

UC-01: Login Pengguna

Kode UC	UC-01
Nama	Login Pengguna (Autentikasi JWT)
Aktor	Pembeli, Admin, Supplier, Kurir
Deskripsi	Pengguna memasukkan email dan password untuk mendapatkan JWT yang digunakan pada seluruh endpoint terproteksi.
Precondition	1. Pengguna terdaftar di tabel users. 2. Password tersimpan format hash bcrypt (cost 14). 3. Endpoint POST /api/v1/auth/login dapat diakses tanpa token.
Main Flow	1. POST /api/v1/auth/login {email, password}. 2. Query SELECT WHERE email=? (parameterized). 3. Jalankan bcrypt.CompareHashAndPassword(). 4. Buat JWT {id_user, role}, TTL 24 jam. 5. Return 200 OK {token, role, expires_in}.
Alt Flow A	Email tidak ditemukan -> 401 Unauthorized (pesan generik).
Alt Flow B	Password salah -> 401 Unauthorized.
Postcondition	Sukses: JWT valid 24 jam. Gagal: Tidak ada token, percobaan dicatat LoggerMiddleware.

UC-02: Checkout dan Penguncian Stok ACID

Kode UC	UC-02
Nama	Checkout dan Penguncian Stok (ACID Transaction)
Aktor	Pembeli (JWT role=pembeli)
Deskripsi	Pembeli menyelesaikan pembelian. Sistem mengunci stok atomik (row-level locking) untuk mencegah overselling, membuat pesanan dan token Midtrans.
Precondition	1. Pembeli login, JWT valid. 2. Keranjang tidak kosong. 3. Stok semua produk mencukupi. 4. Koneksi Midtrans Snap API aktif.
Main Flow	1. POST /api/v1/orders/checkout. 2. tx.Begin() buka transaksi. 3. SELECT stok FOR UPDATE (row-lock). 4. Validasi stok >= kuantitas. 5. Buat record pesanan + pembayaran (PENDING). 6. UPDATE stok produk. 7. Hapus item keranjang. 8. Panggil Midtrans Snap API. 9. COMMIT -> 201 Created + snap_token.
Alt Flow A	Stok kurang -> ROLLBACK total, 400 Bad Request + detail produk.
Alt Flow B	Midtrans error -> ROLLBACK total, 502 Bad Gateway.
Postcondition	Sukses: Pesanan+pembayaran tersimpan, stok berkurang, snap_token aktif. Gagal: Tidak ada perubahan di DB.

UC-03: Rekonsiliasi Pembayaran via Webhook Midtrans

Kode UC	UC-03
Nama	Rekonsiliasi Pembayaran Asinkronus (Webhook Midtrans)
Aktor	Payment Gateway Midtrans (aktor eksternal otomatis)
Deskripsi	Midtrans mengirim notifikasi HTTP POST setelah pembayaran. Sistem memvalidasi tanda tangan SHA-512 dan memperbarui status pesanan atomik.
Precondition	1. Pesanan berstatus PENDING. 2. Server key Midtrans terkonfigurasi. 3. Endpoint POST /api/v1/payments/webhook dapat diakses publik.

Main Flow	1. Midtrans POST /api/v1/payments/webhook. 2. Ekstrak: order_id, status_code, gross_amount, signature_key. 3. Hitung SHA512(order_id+status_code+gross_amount+server_key). 4. Bandingkan dengan signature_key. 5. UPDATE pesanan SET status=PAID. 6. UPDATE pembayaran SET metode, waktu_lunas=NOW(). 7. Return 200 OK.
Alt Flow A	Signature tidak valid -> 403 Forbidden. Status tidak berubah.
Alt Flow B	Pesanan sudah PAID (idempotency) -> skip UPDATE, return 200 OK.
Postcondition	Sukses: status=PAID, waktu_lunas terisi. Gagal: Status tetap PENDING, insiden tercatat.

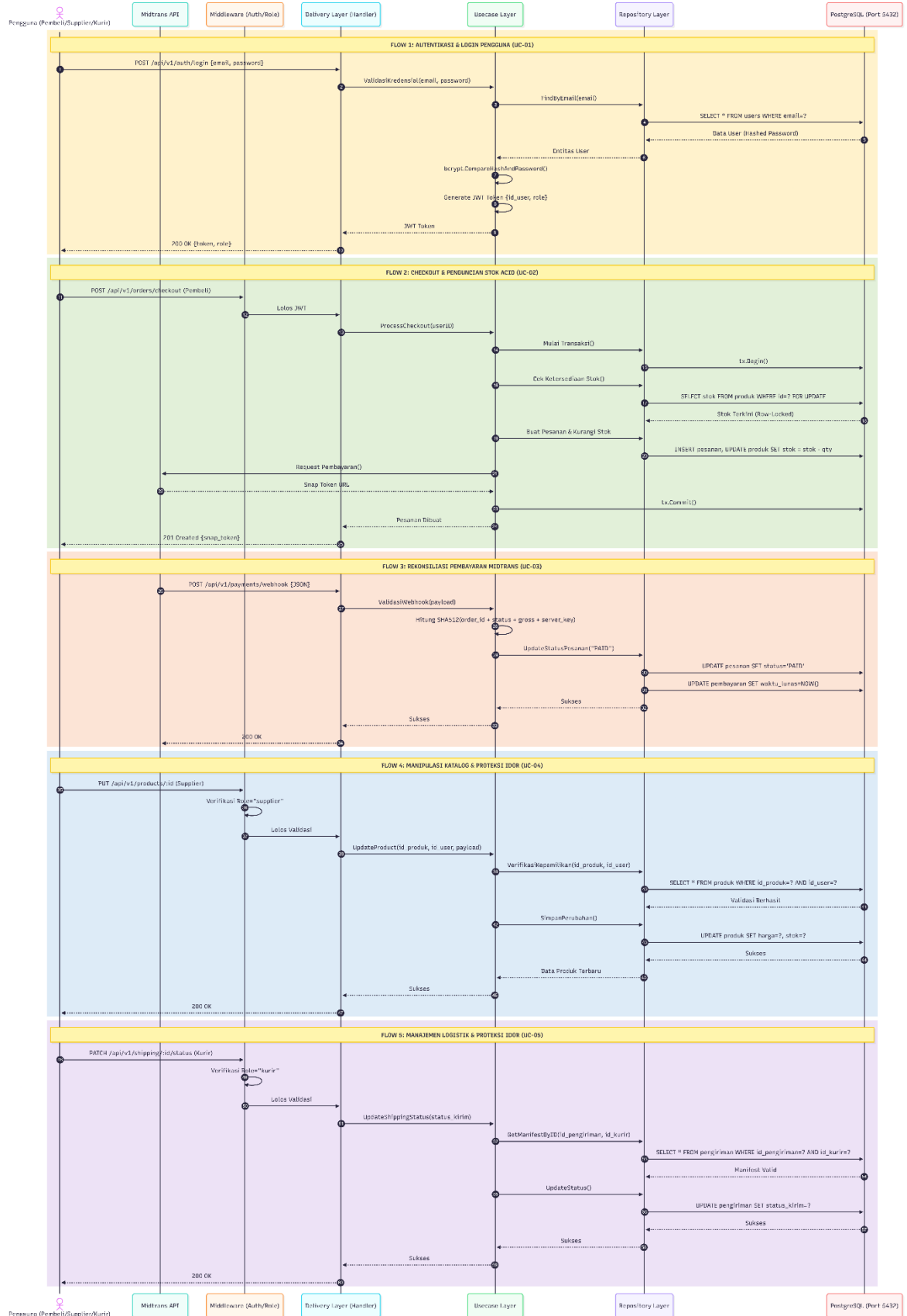
UC-04: Manajemen Produk Supplier (Proteksi IDOR)

Kode UC	UC-04
Nama	Manajemen Produk Supplier (CRUD + Validasi IDOR)
Aktor	Supplier (JWT role=supplier)
Deskripsi	Supplier mendaftarkan/memperbarui/menghapus produk miliknya. Sistem memvalidasi kepemilikan tiap operasi untuk mencegah IDOR.
Precondition	1. Login dengan role=supplier. 2. JWT valid. 3. Untuk UPDATE/DELETE: produk ada di DB.
Main Flow	1. PUT /api/v1/products/:id_produk {nama, harga, stok}. 2. AuthMiddleware: ekstrak id_user dari JWT. 3. RoleMiddleware: validasi role=supplier. 4. SELECT WHERE id_produk=? AND id_user=? (klausa AND id_user kritis). 5. UPDATE data produk. 6. Return 200 OK + data produk terbaru.
Alt Flow A	IDOR: produk bukan milik Supplier ini (langkah 4 nil) -> 403 Forbidden.
Alt Flow B	Validasi input gagal (harga < 0, stok < 0) -> 400 Bad Request.
Postcondition	Sukses: Produk ter-update. Gagal IDOR: Data tidak berubah, upaya tercatat di log audit.

UC-05: Pembaruan Status Pengiriman oleh Kurir

Kode UC	UC-05
Nama	Pembaruan Status Pengiriman (Logistik + Proteksi IDOR)
Aktor	Kurir (JWT role=kurir)
Deskripsi	Kurir yang ditugaskan memperbarui status fisik pengiriman. Sistem memvalidasi Kurir yang request adalah Kurir yang ditugaskan pada manifest.
Precondition	1. Login dengan role=kurir. 2. Manifest pengiriman ada (dibuat setelah PAID). 3. id_kurir di tabel pengiriman sudah diisi Admin.
Main Flow	1. PATCH /api/v1/shipping/:id_pengiriman/status {status_kirim}. 2. AuthMiddleware: ekstrak id_user. 3. RoleMiddleware: validasi role=kurir. 4. SELECT WHERE id_pengiriman=? AND id_kurir=?. 5. Validasi status dalam enum (DIKEMAS/DIKIRIM/DITERIMA). 6. UPDATE status_kirim. 7. Return 200 OK.
Alt Flow A	IDOR: Kurir bukan penanggung manifest -> 403 Forbidden.
Alt Flow B	Status di luar enum -> 400 Bad Request.
Postcondition	Sukses: status_kirim diperbarui. Gagal: Kurir lain tidak bisa memodifikasi manifest ini.

2.2.2 Diagram sequence



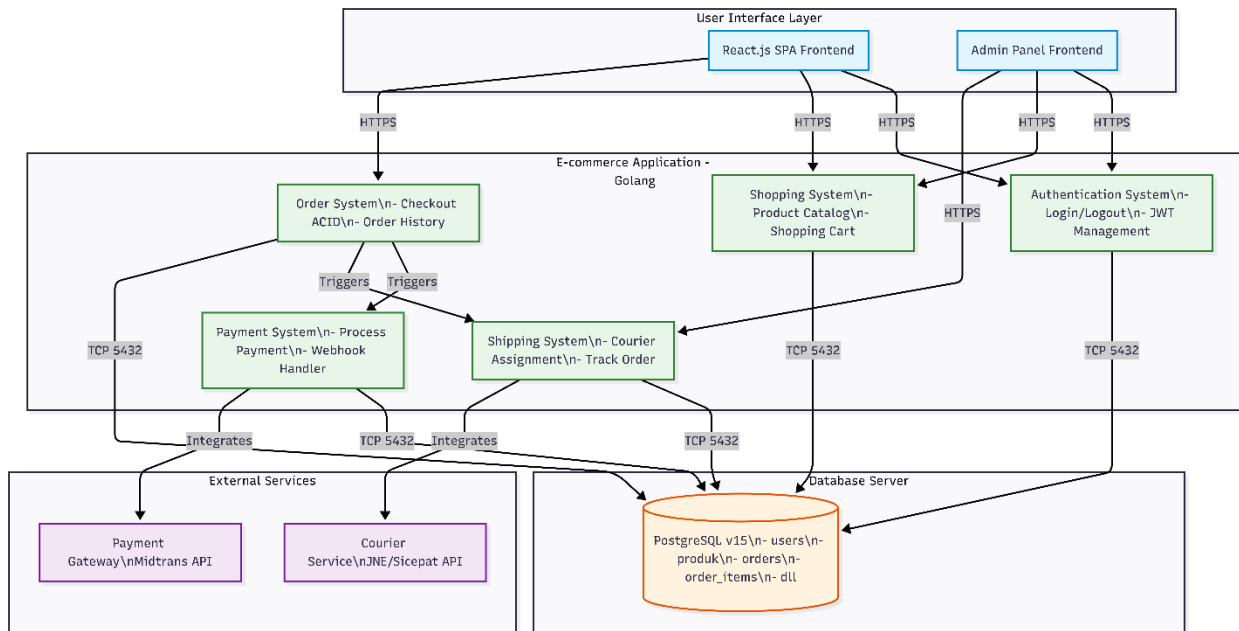
Pemodelan urutan interaksi objek difokuskan pada pembedahan tiga alur pemrosesan yang paling kompleks dan menduduki prioritas utama dalam matriks pengujian jaminan mutu (SQA) .

- Sequence Manipulasi Katalog oleh Supplier: Alur perlindungan objek ini mendemonstrasikan bagaimana peladen mencegah permintaan pembaruan produk dari klien, membongkar identitas asli Supplier dari token JWT, lalu membandingkannya dengan ID Pemilik produk di tabel basis data. Jika tidak identik, sistem secara proaktif menggugurkan proses (*abort*) dengan galat 403 *Forbidden*.
- Sequence Pemesanan ACID (Checkout): Alur transaksional yang mendemonstrasikan pembukaan gerbang blok transaksi di basis data. Sistem mengunci baris stok produk, memvalidasi kecukupan panen, menghitung total nilai keekonomian, menyisipkan data pesanan baru, dan mengeliminasi isi keranjang pembeli. Seluruh rangkaian instruksi ini dieksekusi sebagai satu kesatuan komputasi yang atomik.
- Sequence Validasi Notifikasi Midtrans: Alur keamanan sinkronisasi di mana peladen mengekstrak tanda tangan kriptografi bawaan Midtrans, memadukannya dengan parameter internal, lalu men- *hash* nilainya menggunakan algoritma SHA-512 untuk memastikan bahwa notifikasi pelunasan tersebut mutlak terhindar dari pemalsuan peretas.

2.2.3 Diagram Implementasi

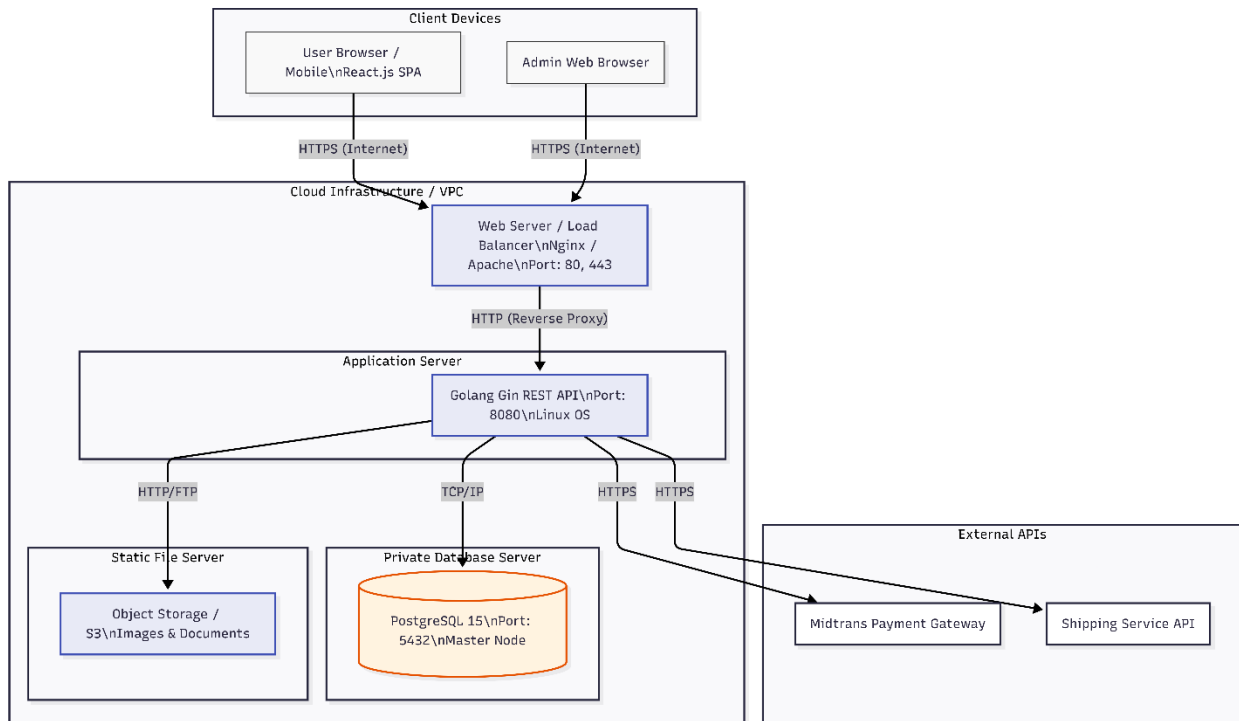
Implementasi fisik arsitektur dijabarkan melalui dua pemodelan diagram topologi teknis guna menegaskan pemisahan fokus (*Separation of Concerns*) secara absolut

A. Diagram komponen



- Pada **Diagram Komponen**, arsitektur kode peladen *backend* dibelah menjadi empat cincin isolasi. Cincin terluar adalah *Delivery* (HTTP Handlers) yang murni memvalidasi lalu lintas JSON. Cincin kedua adalah *Usecase* (Logika Bisnis) tempat otak komputasi berjalan. Cincin ketiga adalah *Repository* yang berfungsi sebagai adaptor kueri GORM. Cincin inti adalah *Domain* yang memuat pendefinisian entitas murni.

B. Diagram deployment



- Pada **Diagram Deployment (Penempatan)**, topologi sistem mendistribusikan beban ke berbagai simpul komputasi: Peramban klien merender SPA React.js, peladen *Application Server* di *cloud* mengeksekusi *binary runtime* Golang di porta 8080, dan *Database Server* ditempatkan pada jaringan privat tertutup (VPC) yang mengeksekusi mesin PostgreSQL di porta 5432.

2.3 Model Data

Perancangan arsitektur pemodelan data pada sistem disusun untuk merajut ekosistem relasional yang mampu memfasilitasi arus informasi dari hulu (suplai) ke hilir (pengiriman) secara konsisten, tanpa memicu anomali duplikasi data

2.3.1 Definisi Domain/Type

Bahasa pemrograman Golang beroperasi dengan pengetikan statis yang sangat kaku, sedangkan PostgreSQL memiliki variasi spektrum tipe data relasional yang luas. Untuk itu, sistem mewajibkan pemetaan kamus tipe data secara eksplisit guna menangkal galat alokasi memori

Nama Domain	Format di Golang	Format di PostgreSQL	Keterangan Teknis Penggunaan
Identitas Baris	string	VARCHAR(36)	Standar UUID untuk mencegah tabrakan ID lintas-aktor.
Nilai Finansial	float64	NUMERIC(15,2)	Presisi ganda agar tagihan bebas dari galat pembulatan desimal.
Satuan Barang	int	INTEGER	Mewakili entitas fisik sayuran yang komputasinya tidak bisa dipecah.
Jejak Waktu	time.Time	TIMESTAMP	Perekaman waktu audit absolut (presisi hingga milidetik).

2.3.2 Data Model Logika

Berdasarkan implementasi aktual pada kode sumber (domain/*.go), model data logika memiliki deviasi dari rancangan awal SKPL. Deviasi utama adalah penggabungan tabel pembayaran dan tabel pengiriman ke dalam tabel orders sebagai keputusan desain untuk menyederhanakan arsitektur. Selain itu, terdapat 8 entitas tambahan yang diimplementasikan di luar spesifikasi SKPL.

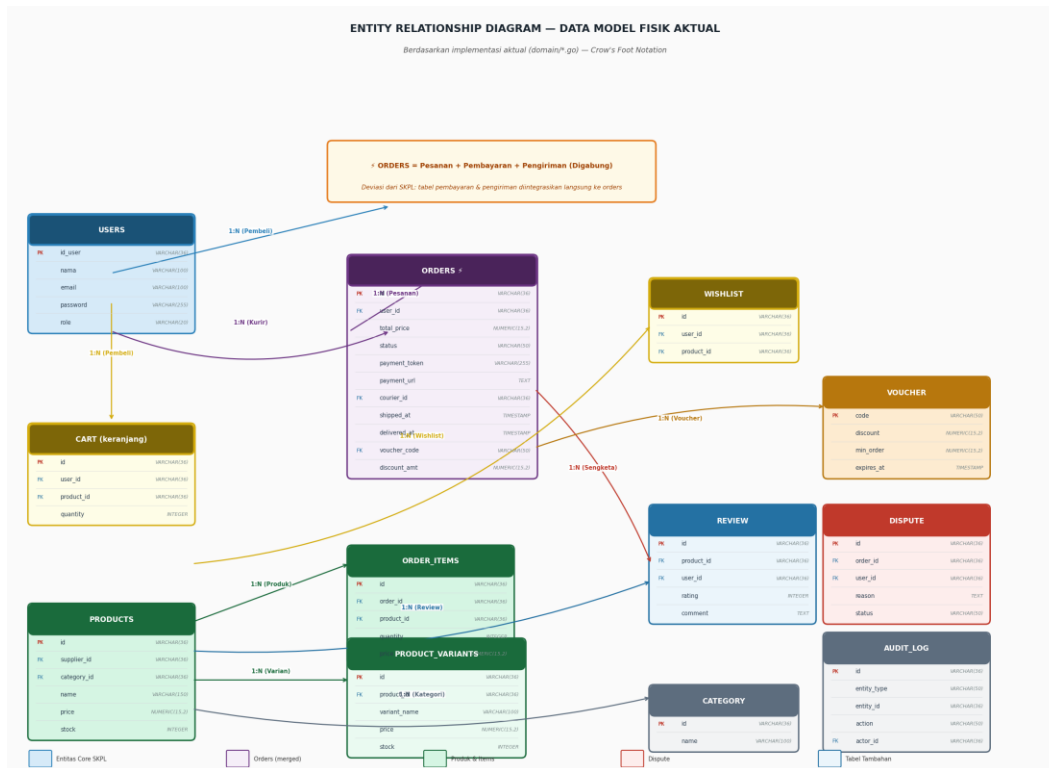
Deviasi Desain Tabel (dibandingkan SKPL):

1. Tabel orders menggantikan tabel pesanan + pembayaran + pengiriman. Field payment_token, payment_url (integrasi Midtrans), courier_id, shipped_at, dan delivered_at disimpan langsung di tabel orders — bukan di tabel terpisah.
2. Tabel order_items menyimpan detail item per pesanan sebagai relasi 1:N dengan orders (junction table antara orders dan products).

Entitas Tambahan di Luar SKPL (bonus implementasi):

category (kategori produk), product_variants (varian produk: ukuran/jenis), voucher (kode diskon), dispute (sengketa/komplain pembeli), audit_log (log perubahan entitas), wishlist (daftar keinginan pembeli), review (ulasan produk oleh pembeli).

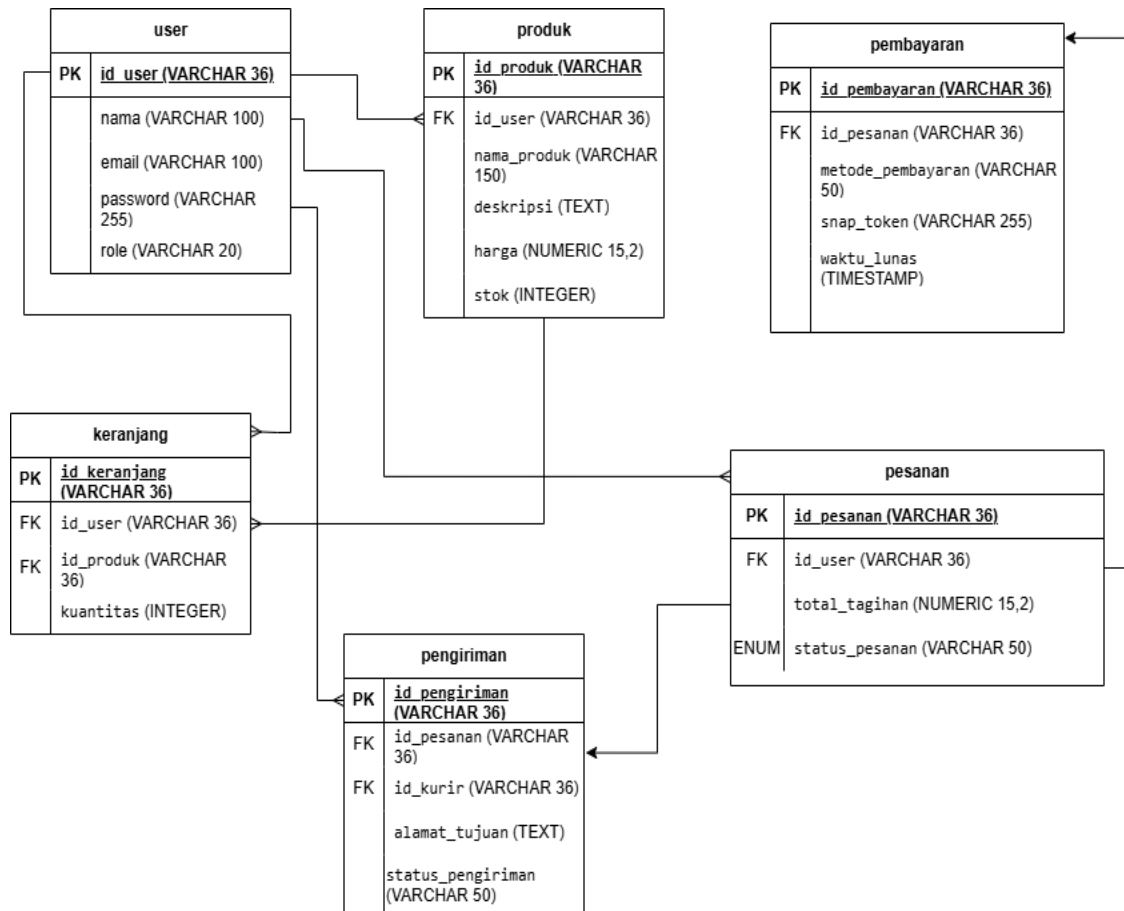
ERD berikut menggambarkan model data fisik aktual berdasarkan domain/*.go:



Gambar 2.3.2 — ERD Aktual: Orders Tergabung (Crow's Foot Notation)

2.3.3 Data Model Fisik

Model data fisik adalah manifestasi teknis dari rancangan logika ke dalam skema struktur tabel di mesin PostgreSQL . Konfigurasi fisik ini menegaskan integritas referensial melalui batasan Kunci Tamu (*Foreign Key Constraints*) secara radikal. Sebagai contoh, peladen melarang keras eksekusi penghapusan terhadap sebuah entitas produk apabila produk tersebut statusnya masih bertengger di dalam tabel pengiriman yang belum selesai diantar oleh kurir (*Restrict Delete Cascade*). Pada lapisan optimasi, model fisik ini menerapkan teknik *B-Tree Indexing* pada kolom alamat surel untuk memastikan waktu pencarian data kredensial tidak melebihi ambang batas puluhan milidetik.



2.3.4 daftar tabel aplikasi

Rincian katalog relasional di bawah ini dibangun secara otomatis oleh mesin migrasi GORM untuk menopang keseluruhan siklus operasional komersial dan skenario pengujian aplikasi

Nama Tabel	Primary Key	Data Store	E/R	Spesifikasi Deskripsi Isi Tabel
users	id_user	PostgreSQL	Ya	Penyimpan profil sentral seluruh aktor. Dilindungi kriptografi sandi mutakhir (<i>bcrypt</i>).
produk	id_produk	PostgreSQL	Ya	Etalase komersial berisi rincian harga, stok aktual panen, dan tautan kepemilikan Supplier.
keranjang	id_keranjang	PostgreSQL	Ya	Penahan antrean transaksional sementara pembeli sebelum menuju kalkulasi <i>checkout</i> .

pesanan	id_pesanan	PostgreSQL	Ya	Arsip komitmen transaksi absolut yang merekam tagihan finansial dan perjalanan status.
pembayaran	id_pembayaran	PostgreSQL	Ya	Log rekonsiliasi finansial yang beroperasi asinkronus menyinkronkan respons peladen Midtrans.
pengiriman	id_pengiriman	PostgreSQL	Ya	Entitas manifest logistik yang mengikat alamat pembeli dengan pergerakan pelacakan kurir.

III. DESKRIPSI PERANCANGAN RINCI

Bagian ini menguraikan secara komprehensif spesifikasi struktur penyimpanan fisik pada tingkat basis data, rancangan kontrak antarmuka pemrograman aplikasi (API), serta algoritma pemrosesan logika bisnis yang dieksekusi oleh peladen *backend* Golang . Mengingat arsitektur sistem ini mengadopsi pemisahan (*decoupled*) antara *Frontend* dan *Backend*, spesifikasi komponen layar visual (*UI components*) direpresentasikan melalui spesifikasi titik akhir jaringan (*API Endpoints*), muatan data (*JSON Payload*), dan perantara keamanan (*Middleware*). Perancangan rinci ini menjadi pedoman mutlak bagi pengembang sumber kode dan acuan bagi insinyur jaminan mutu (QA) dalam menyusun matriks pengujian fungsional dan keamanan (SQA).

3.1 Deskripsi Rinci Tabel

Setiap tabel yang telah didefinisikan pada rancangan arsitektur data global (Bab II) dirinci satu per satu pada bagian ini untuk menjelaskan tipe data fisik, panjang karakter alokasi memori, serta aturan batasan (*constraints*) yang ditegakkan secara absolut oleh mesin PostgreSQL .

3.1.1 Tabel User

Identifikasi>Nama: users

Deskripsi Isi: Tabel sentral yang bertugas mengamankan profil kredensial seluruh entitas pengguna (Admin, Pembeli, Supplier, dan Kurir). Atribut sandi disamarkan menggunakan kriptografi *bcrypt* (*cost* 14) untuk mencegah kebocoran teks terang (*plaintext*).

Jenis: Tabel Data Induk Kredensial

Volume: Diperkirakan 10.000+ baris data. **Laju:** Moderat, estimasi 50-100 pendaftaran per hari.

Primary Key: id_user

Id Field	Deskripsi	Tipe & Length	Boleh NULL	Default	Keterangan
id_user	Identitas unik	VARCHAR(36)	NO	UUIDv4	Mencegah eksploitasi IDOR.
nama	Nama lengkap	VARCHAR(100)	NO	-	Wajib diisi pengguna.
email	Alamat surel	VARCHAR(100)	NO	-	Indeks Unik (B-Tree).
password	Sandi enkripsi	VARCHAR(255)	NO	-	Hasil komputasi <i>bcrypt</i> .

role	Hak akses otorisasi	VARCHAR(20)	NO	'pembeli'	Nilai: pembeli, admin, supplier, kurir.
------	---------------------	-------------	----	-----------	---

3.1.2 Tabel Produk

Identifikasi>Nama: Produk

Deskripsi Isi: Tabel katalog komersial yang menyimpan rincian harga, deskripsi sayuran, dan pelacakan kuantitas stok panen yang diperbarui secara langsung oleh Supplier di hulu rantai pasok.

Volume: Sedang, berkisar 1.000 - 5.000 baris.

Jenis: Tabel Data Induk dan Dinamis

Laju: Pembaruan kolom stok terjadi sangat cepat mengikuti arus pesanan.

Primary Key: id_produk

Spesifikasi Kolom Tabel Produk:

ID Field	Deskripsi	Tipe & Length	Boleh NULL	Default	Keterangan
id_produk	Identitas sayuran	VARCHAR(36)	NO	UUIDv4	Kunci primer UUID.
id_user	Pemilik (Supplier)	VARCHAR(36)	NO	-	<i>Foreign Key</i> ke users.
nama_produk	Judul komoditas	VARCHAR(150)	NO	-	Deskripsi nama sayuran.
deskripsi	Info tambahan	TEXT	YES	NULL	Penjelasan spesifikasi/panen.
harga	Harga jual akhir	NUMERIC(15,2)	NO	-	Presisi mata uang mutlak.
stok	Ketersediaan	INTEGER	NO	0	Dilarang bernilai minus (< 0).

3.1.3 Tabel Keranjang

- **Identifikasi>Nama:** keranjang
- **Deskripsi Isi:** Tabel transaksional sementara yang bertindak sebagai antrean ruang tunggu. Menampung rincian produk yang dipilih pembeli sebelum mereka mengeksekusi perjanjian transaksi finansial final.

- **Jenis:** Tabel Data Transaksi Sementara
- **Volume:** Tinggi dan sangat fluktuatif.
- **Laju:** Sangat cepat untuk komputasi penyisipan dan penghapusan fisik.
- **Primary Key:** id_keranjang

Id Field	Deskripsi	Tipe & Length	Boleh NULL	Default	Keterangan
id_keranjang	Identitas antrean	VARCHAR(36)	NO	UUIDv4	Kunci primer UUID.
id_user	Pemilik keranjang	VARCHAR(36)	NO	-	<i>Foreign Key</i> ke users (Pembeli).
id_produk	Sayuran terpilih	VARCHAR(36)	NO	-	<i>Foreign Key</i> ke produk.
kuantitas	Jumlah barang	INTEGER	NO	1	Minimal bernilai 1.

3.1.4 Tabel Pesanan

- Identifikasi>Nama: pesanan
- Deskripsi Isi: Tabel arsip permanen yang mencatat riwayat akhir transaksi berstandar ACID. Menyimpan komitmen total tagihan pembayaran dan menjejaki status makro dari perjalanan komoditas.
- Jenis: Tabel Data Transaksi Permanen
- Volume: Tinggi, diproyeksikan 50.000+ baris per bulan.
- Laju: Pertumbuhan akumulatif cepat tanpa ada penghapusan data.
- Primary Key: id_pesanan

Id Field	Deskripsi	Tipe & Length	Boleh NULL	Default	Keterangan
----------	-----------	---------------	------------	---------	------------

id_pesanan	Nomor resi transaksi	VARCHAR(36)	NO	UUIDv4	Identitas pesanan global.
id_user	Pemilik pesanan	VARCHAR(36)	NO	-	<i>Foreign Key</i> ke users (Pembeli).
total_tagihan	Nominal bersih	NUMERIC(15,2)	NO	-	Kalkulasi absolut oleh peladen.
status_pesanan	Transisi status	VARCHAR(50)	NO	'PENDING'	Enum: PENDING (menunggu bayar) / PAID (lunas) / CANCELLED (dibatalkan) / EXPIRED (kedaluwarsa).

3.1.5 Tabel Pembayaran (Diintegrasikan ke Tabel orders)

Identifikasi>Nama: orders (kolom pembayaran)

Deviasi Implementasi: Berdasarkan review kode sumber aktual (domain/order.go), data pembayaran TIDAK disimpan dalam tabel terpisah. Field terkait pembayaran diintegrasikan langsung ke dalam tabel orders sebagai keputusan desain arsitektur untuk menyederhanakan relasi database.

Kolom terkait pembayaran di tabel orders:

- payment_token VARCHAR(255): Token sesi antarmuka Midtrans Snap (setara snap_token di SKPL).
- payment_url TEXT: URL redirect ke halaman pembayaran Midtrans.
- status VARCHAR(50): Mencakup status pembayaran sekaligus pengiriman (PENDING, PAID, CANCELLED, PROCESSED, SHIPPED, DELIVERED).

Catatan: Relasi 1:1 antara pesanan dan pembayaran tetap terjaga karena field ini merupakan bagian dari entitas orders itu sendiri.

3.1.6 Tabel Pengiriman (Diintegrasikan ke Tabel orders)

Identifikasi>Nama: orders (kolom pengiriman)

Deviasi Implementasi: Berdasarkan review kode sumber aktual (domain/order.go), data pengiriman TIDAK disimpan dalam tabel terpisah. Field terkait logistik diintegrasikan langsung ke dalam tabel orders.

Kolom terkait pengiriman di tabel orders:

- courier_id VARCHAR(36): Foreign Key ke tabel users (Kurir yang ditugaskan). Nullable sampai Admin menugaskan Kurir.

- shipped_at TIMESTAMP: Timestamp saat Kurir mengambil dan mengirim barang. Setara dengan transisi status ke SHIPPED.
- delivered_at TIMESTAMP: Timestamp saat barang diterima pembeli. Setara dengan transisi status ke DELIVERED.

Catatan: Status pengiriman tidak memiliki enum terpisah (DIKEMAS/DIKIRIM/DITERIMA) seperti di SKPL. Status direpresentasikan oleh nilai di kolom status orders: PROCESSED (sedang dikemas), SHIPPED (dikirim), DELIVERED (diterima).

3.2 Deskripsi Proses Secara Rinci

Bagian ini menguraikan setiap antarmuka pemrograman aplikasi (API) sesuai dengan pemodelan aliran sistem pada rancangan global . Spesifikasi layar (*UI Specification*) dalam dokumen ini diproyeksikan sebagai muatan data jaringan (*Network Payload*) untuk menyesuaikan standar rekayasa aplikasi web modern.

3.2.1 Spesifikasi Proses: Autentikasi Pengguna (Login API)

Identifikasi>Nama: Modul Pintu Masuk dan Keamanan Sesi Berbasis Token **Deskripsi Isi:** Titik akhir (*endpoint*) yang memfasilitasi pengguna untuk memvalidasi identitas. Peladen membandingkan masukan dengan enkripsi *bcrypt* dan menerbitkan sesi *JSON Web Token* (JWT) . **Jenis:** RESTful API Endpoint HTTP

3.2.1.1 Spesifikasi Tabel Input

- Tabel users (Murni operasi *Read-Only* untuk pencocokan data).

3.2.1.2 Spesifikasi Tabel Output

- TIDAK ADA (Tidak ada manipulasi atau penambahan data baru di basis data).

3.2.1.3 Spesifikasi Layar Utama (Endpoint API) Mengingat arsitektur terpisah (*Decoupled*), layar utama direpresentasikan oleh titik akses jaringan:

- **Metode Jaringan:** POST
- **Rute URL:** /api/v1/auth/login
- **Otorisasi:** Rute publik (Bebas diakses tanpa token JWT).

3.2.1.4 Spesifikasi Query

- GORM Query: `SELECT * FROM users WHERE email = ? LIMIT 1;` (Parameter ? mencegah Injeksi SQL)

3.2.1.5 Spesifikasi Field Data Pada Layar (JSON Payload)

Klien mengirimkan muatan data masukan sebagai berikut:

- email: String (Wajib memuat format surel sah).
- password: String (Wajib, panjang minimum 8 karakter keamanan)

3.2.1.6 Spesifikasi Function Key / Objek Pada Layar (Middleware)

Fungsi tombol atau objek kontrol direpresentasikan oleh perantara sistem (*Middleware*):

- **LoggerMiddleware:** Komponen yang mencatat alamat IP pengguna dan waktu percobaan masuk untuk keperluan audit pelacakan anomaly.

3.2.1.7 Spesifikasi Layar Pesan (API Response)

Contoh Response Sukses (200 OK):

```
{
  "status": "success",
  "message": "Login berhasil",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "role": "pembeli",
    "expires_in": "24h"
  }
}
```

Contoh Response Gagal (401 Unauthorized):

```
{
  "status": "error",
  "message": "Email atau password salah"
}
```

- **Pesan Sukses:** Mengembalikan kode HTTP 200 OK dengan format JSON berisi atribut token JWT.
- **Pesan Gagal:** Mengembalikan kode HTTP 401 Unauthorized jika kredensial salah atau tidak ditemukan.

3.2.1.8 Spesifikasi Proses / Algoritma

1. **Initial State (IS):** Ekstraksi muatan permintaan JSON (email, password) dari klien.
2. Lakukan kueri pencarian melalui GORM ke tabel users.
3. IF hasil nihil, hentikan rantai eksekusi dan kembalikan galat 401 Unauthorized.
4. ELSE (Surel ditemukan), eksekusi komputasi `bcrypt.CompareHashAndPassword()`.
5. IF validasi kriptografi sandi gagal, kembalikan 401 Unauthorized.
6. Tanamkan identitas (id_user) dan kewenangan (role) ke dalam muatan klaim JWT.
7. Tandatangani menggunakan Kunci Rahasia (*Secret Key*) peladen dan atur batas kedaluwarsa 24 jam.
8. **Final State (FS):** Terbitkan JWT kepada klien dalam format balasan JSON (200 OK).

3.2.1.9 Spesifikasi State Chart

- TIDAK ADA (Sistem bersifat *stateless* dan tidak memiliki perilaku transisi status antarmuka visual yang kompleks).

3.2.1.10 Spesifikasi Report (Matriks Pengujian SQA)

- **Test Case (AUTH-01):** Percobaan masuk masif (*Brute-force*).
- **Ekspektasi Sistem:** Waktu respon algoritma *bcrypt* terbukti konstan di atas ambang batas 500ms untuk melumpuhkan serangan tebakan peretas.

No TC	Skenario Uji	Precondition	Input / Aksi	Expected Result	Pass Criteria
AUTH-01	Login berhasil (Happy Path)	User terdaftar di DB	POST /api/v1/auth/login { "email": "user@test.com", "password": "Pass@1234" }	HTTP 200 OK + JWT token	Status 200, ada field "token"
AUTH-02	Login gagal — password salah	User terdaftar di DB	POST /api/v1/auth/login { "email": "user@test.com", "password": "salah123" }	HTTP 401 Unauthorized	Status 401, tidak ada token
AUTH-03	Login gagal — email tidak ada	Tidak ada akun dengan email ini	POST /api/v1/auth/login { "email": "tidakada@test.com", "password": "Pass@1234" }	HTTP 401 Unauthorized	Status 401, pesan generik

AUTH-04	Akses endpoint tanpa token	Server berjalan normal	GET /api/v1/products/123 tanpa header Authorization	HTTP 401 Unauthorized dari AuthMiddleware	Status 401, request tidak mencapai handler
AUTH-05	Token expired	JWT sudah melewati TTL 24 jam	GET /api/v1/orders dengan token expired	HTTP 401 Unauthorized	Status 401, pesan "token expired"

3.2.2 Spesifikasi Proses: Manajemen Katalog & Kepemilikan (Khusus Supplier)

Identifikasi>Nama: Modul Ekosistem Hulu dan Validasi *Insecure Direct Object Reference* (IDOR)

Deskripsi Isi: Proses di mana aktor Supplier mengatur rincian panen mereka. Modul ini menerapkan proteksi lapis ganda validasi identitas .

Jenis: RESTful API Endpoint Transaksional

- 3.2.2.1 Spesifikasi Tabel Input: Tabel produk.
- 3.2.2.2 Spesifikasi Tabel Output: Tabel produk (Menjalankan operasi pembaruan dan penghapusan).
- 3.2.2.3 Spesifikasi Layar Utama (Endpoint API): Metode PUT dan DELETE menuju rute /api/v1/products/:id_produk.
- 3.2.2.4 Spesifikasi Query: UPDATE produk SET harga = ?, stok = ? WHERE id_produk = ? AND id_user = ?;
- 3.2.2.5 Spesifikasi Field Data Pada Layar (JSON Payload): nama_produk (String), harga (Numerik > 0), stok (Integer >= 0).
- 3.2.2.6 Spesifikasi Function Key (Middleware): AuthMiddleware dan RoleMiddleware khusus "supplier".
- 3.2.2.7 Spesifikasi Layar Pesan (API Response): Mengembalikan HTTP 200 OK jika berhasil.

Contoh Response Sukses Update Produk (200 OK):

```
{
  "status": "success",
  "data": {
    "id_produk": "uuid-xxx",
```



```

    "nama_produk": "Bayam Hijau Segar",
    "harga": 15000,
    "stok": 100
  }
}

```

Contoh Response IDOR (403 Forbidden):

```

{
  "status": "error",
  "message": "Akses ditolak: Anda bukan pemilik produk ini"
}

```

- 3.2.2.8 Spesifikasi Proses / Algoritma: 1) Validasi Role "supplier". 2) Ekstrak id_user dari JWT. 3) Cari target id_produk. 4) Jika pemilik tabel tidak cocok dengan sesi (IDOR), lempar 403 Forbidden. 5) Jika cocok, perbarui data.
- 3.2.2.9 Spesifikasi State Chart: TIDAK ADA.
- 3.2.2.10 Spesifikasi Report (Matriks Pengujian SQA): Test Case CAT-01 (Uji IDOR lintas Supplier).

No TC	Skenario Uji	Precondition	Input / Aksi	Expected Result	Pass Criteria
CAT-01	Update produk sendiri (Happy Path)	Supplier login, id_produk miliknya	PUT /api/v1/products/uuid-milik-sendiri {"harga":15000,"stok":100}	HTTP 200 OK + data produk baru	Status 200, data stok dan harga terupdate
CAT-02	Update produk milik Supplier lain (IDOR)	Supplier A login, id_produk milik Supplier B	PUT /api/v1/products/uuid-milik-supplierB {"harga":5000}	HTTP 403 Forbidden	Status 403, data produk Supplier B tidak berubah
CAT-03	Buat produk baru (Happy Path)	Supplier login	POST /api/v1/products {"nama_produk":"Kangkung","harga":8000,"stok":200}	HTTP 201 Created + data produk baru	Status 201, ada id_produk baru di response
CAT-04	Hapus produk milik sendiri	Supplier login, id_produk miliknya	DELETE /api/v1/products/uuid-milik-sendiri	HTTP 200 OK	Status 200, produk tidak ada lagi di DB

3.2.3 Spesifikasi Proses: Checkout dan Penguncian Stok ACID (Khusus Pembeli)

Identifikasi/Nama: Modul Resolusi Transaksi Komersial ACID

Deskripsi Isi: Modul finansial ekstrem yang mengubah antrean keranjang menjadi pesanan riil menggunakan *Row-level Locking* . Jenis: RESTful API Endpoint Komputasi Berat

- 3.2.3.1 Spesifikasi Tabel Input: Tabel keranjang dan produk.

- 3.2.3.2 Spesifikasi Tabel Output: Tabel pesanan, pembayaran (Insert), keranjang (Delete), produk (Update).
- 3.2.3.3 Spesifikasi Layar Utama (Endpoint API): Metode POST rute `/api/v1/orders/checkout`.
- 3.2.3.4 Spesifikasi Query: `SELECT stok FROM produk WHERE id_produk = ? FOR UPDATE;`
- 3.2.3.5 Spesifikasi Field Data Pada Layar (JSON Payload): TIDAK ADA (Identitas mutlak diambil dari JWT Header).
- 3.2.3.6 Spesifikasi Function Key (Middleware): `RecoveryMiddleware` (mencegah *server crash*).
- 3.2.3.7 Spesifikasi Layar Pesan (API Response): HTTP 201 Created beserta token Midtrans.

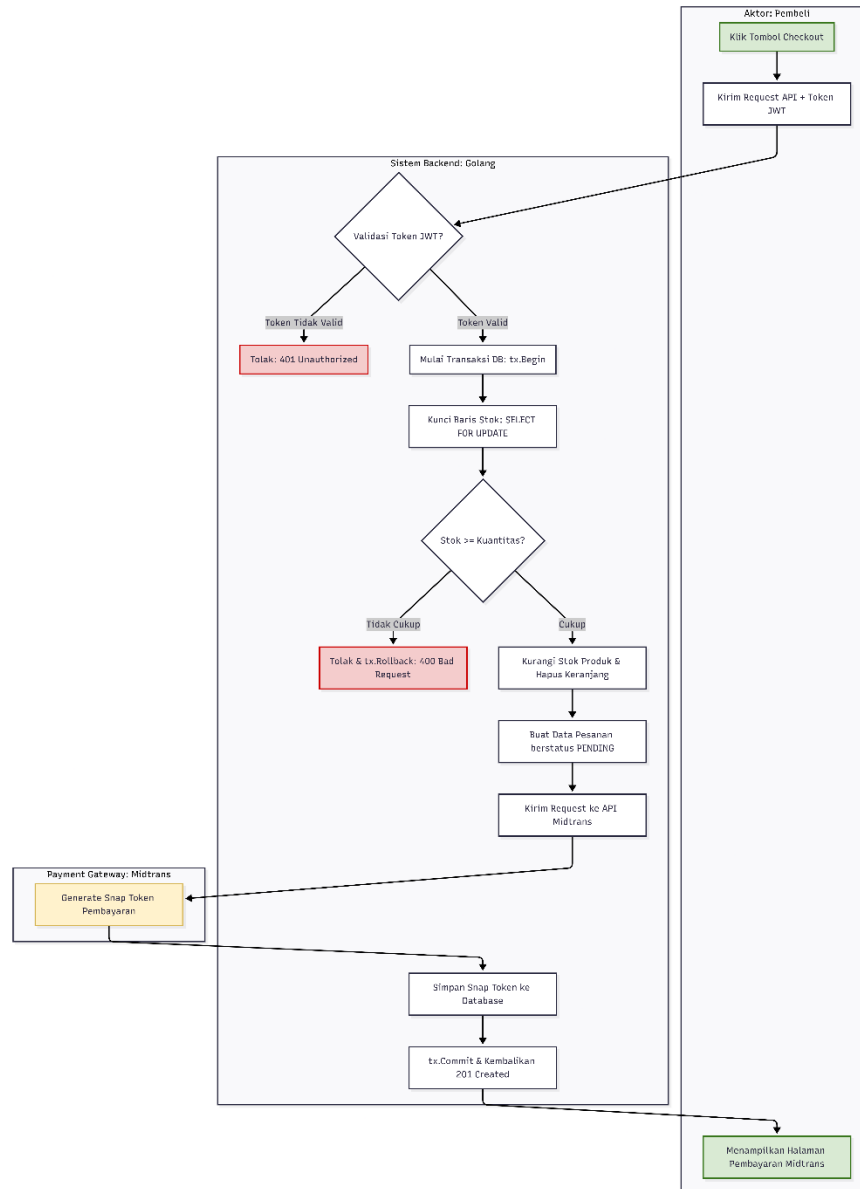
Contoh Response Sukses Checkout (201 Created):

```
{
  "status": "success",
  "data": {
    "id_pesanan": "uuid-xxx",
    "total_tagihan": 75000,
    "status_pesanan": "PENDING",
    "snap_token": "snap-token-midtrans-xxx",
    "redirect_url": "https://app.midtrans.com/snap/v2/vtweb/snap-token"
  }
}
```

Contoh Response Stok Habis (400 Bad Request):

```
{
  "status": "error",
  "message": "Stok produk tidak mencukupi",
  "data": {
    "stok_tersedia": 2,
    "stok_diminta": 5
  }
}
```

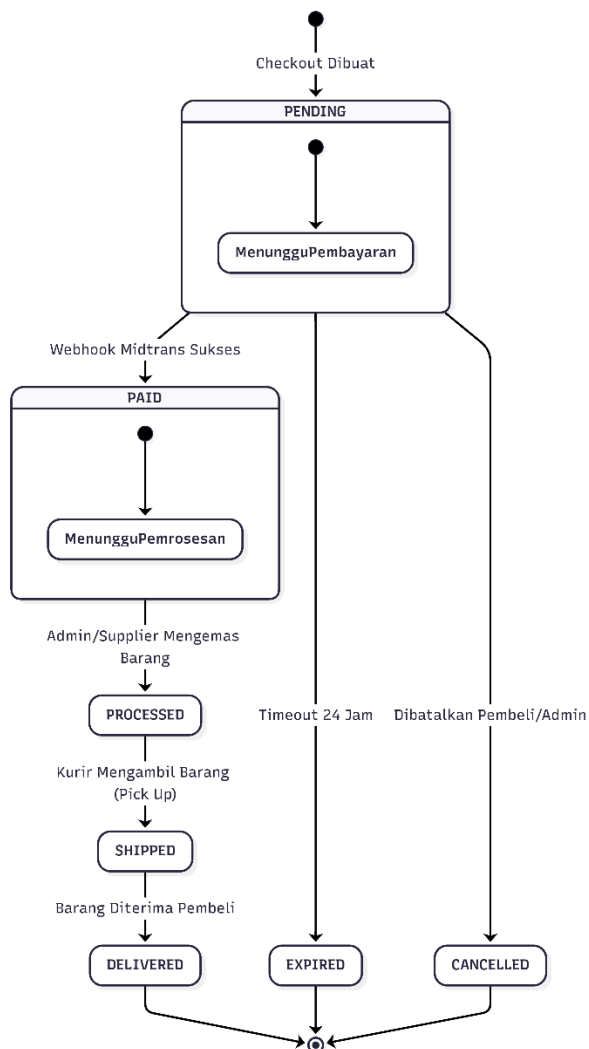
- 3.2.3.8 Spesifikasi Proses / Algoritma: 1) Mulai `tx.Begin()`. 2) Kueri `FOR UPDATE`. 3) Jika stok kurang, `tx.Rollback()`. 4) Kalkulasi total. 5) Minta token Midtrans. 6) Simpan Pesanan. 7) Hapus Keranjang. 8) `tx.Commit()`.



Gambar 3.X Diagram Activity Alur Checkout

• 3.2.3.9 Spesifikasi State Chart

Bagian ini menjabarkan pemodelan *State Chart* yang memvisualisasikan siklus hidup dinamis (*lifecycle*) dari objek **Pesanan (Order)**. Karena aplikasi ini mengimplementasikan transaksi asinkronus dengan pihak ketiga (Midtrans) dan melibatkan alur logistik fisik, diagram ini krusial untuk memetakan bagaimana status transaksi berubah dari satu fase ke fase lainnya berdasarkan pemicu (*event/trigger*) tertentu.



Gambar 3.X - State Chart Transisi Status Pesanan

Keterangan Transisi Status:

1. **PENDING (Menunggu Pembayaran):** Status awal / *Initial State* yang otomatis disematkan oleh sistem Golang sesaat setelah pembeli berhasil melakukan *checkout* dan *snap token* Midtrans dibuat.
2. **PAID (Lunas):** Transisi yang dipicu secara otomatis oleh peladen (otomasi *server-to-server*) melalui penerimaan *Webhook* Midtrans yang sukses tervalidasi.
3. **PROCESSED (Diproses):** Status yang diperbarui oleh Admin atau Supplier sebagai penanda bahwa sayuran sedang dikemas.
4. **SHIPPED (Dikirim):** Transisi yang dipicu ketika Kurir melakukan *pick up* dan memperbarui status pengiriman via API.
5. **DELIVERED (Diterima):** Status akhir (*Final State*) sukses yang menandakan barang fisik telah diterima oleh pembeli.

6. **EXPIRED / CANCELLED:** Status akhir (*Final State*) gagal. EXPIRED terpicu otomatis jika sistem mendeteksi *timeout* 24 jam tanpa pembayaran, sedangkan CANCELLED terpicu oleh intervensi pembatalan manual dari Pembeli atau Admin.

- 3.2.3.10 Spesifikasi Report (Matriks Pengujian SQA): Test Case CHK-01 (Pengujian stres konkurensi / *Race Condition*).

No TC	Skenario Uji	Precondition	Input / Aksi	Expected Result	Pass Criteria
CHK-01	Checkout berhasil (Happy Path)	Pembeli login, keranjang terisi, stok cukup	POST /api/v1/orders/checkout	HTTP 201 Created + snap_token	Status 201, stok berkurang, keranjang kosong
CHK-02	Checkout gagal — stok tidak cukup	Pembeli login, stok produk = 2, kuantitas = 5	POST /api/v1/orders/checkout	HTTP 400 Bad Request + detail stok	Status 400, stok tidak berubah, pesanan tidak dibuat
CHK-03	Race condition — concurrent checkout	2 pembeli checkout produk terakhir bersamaan	2x POST /api/v1/orders/checkout (concurrent)	Salah satu 201, satunya 400 (stok habis)	Tidak ada overselling (stok >= 0)
CHK-04	Checkout tanpa login	Tidak ada JWT	POST /api/v1/orders/checkout tanpa Authorization	HTTP 401 Unauthorized	Status 401, tidak ada pesanan dibuat

3.2.4 Spesifikasi Proses: Rekonsiliasi Pembayaran (Webhook Midtrans)

Identifikasi/Nama: Modul Rekonsiliasi Finansial Asinkronus Deskripsi Isi: Otomasi *Server-to-Server* di mana Midtrans mengirimkan HTTP POST. Dilindungi validasi SHA-512 . Jenis:

RESTful API Endpoint (Webhook)

- 3.2.4.1 Spesifikasi Tabel Input: TIDAK ADA (Muatan dari eksternal).
- 3.2.4.2 Spesifikasi Tabel Output: Tabel pesanan dan pembayaran (Pembaruan status).
- 3.2.4.3 Spesifikasi Layar Utama (Endpoint API): Metode POST rute
/api/v1/payments/webhook.
- 3.2.4.4 Spesifikasi Query: UPDATE pesanan SET status_pesanan = 'PAID' WHERE id_pesanan = ?;
- 3.2.4.5 Spesifikasi Field Data Pada Layar (JSON Payload): order_id, status_code, gross_amount, signature_key.
- 3.2.4.6 Spesifikasi Function Key (Middleware): RecoveryMiddleware.
- 3.2.4.7 Spesifikasi Layar Pesan (API Response): HTTP 200 OK (Penerimaan mutlak).

Contoh Response Webhook Diterima (200 OK):

```
{
  "status": "success",
```

```

    "data": {
      "id_pesanan": "uuid-xxx",
      "status_pesanan": "PAID",
      "metode": "gopay",
      "waktu_lunas": "2026-03-08T10:30:00Z"
    }
  }
}

```

Contoh Response Signature Invalid (403):

```

{
  "status": "error",
  "message": "Signature key tidak valid"
}

```

- 3.2.4.8 Spesifikasi Proses / Algoritma: 1) Terima JSON Midtrans. 2) Rangkai order_id+status+gross+ServerKey. 3) Hash SHA-512. 4) Bandingkan dengan signature_key. 5) Jika beda tolak. 6) Jika sama perbarui status pesanan menjadi PAID.

State Chart modul Pembayaran mengikuti siklus yang sama dengan Pesanan (lihat **3.3 Dekomposisi Fisik Modul**). Secara spesifik: record tabel pembayaran dibuat saat PENDING dengan snap_token Midtrans. Kolom waktu_lunas dan metode diisi saat transisi ke PAID via Webhook. Handler bersifat idempotent: jika Webhook diterima ganda untuk pesanan yang sudah PAID, sistem mengembalikan 200 OK tanpa duplikasi record.

- 3.2.4.10 Spesifikasi Report (Matriks Pengujian SQA): Test Case PAY-01 (Simulasi penolakan *Signature Key* palsu).

No TC	Skenario Uji	Precondition	Input / Aksi	Expected Result	Pass Criteria
PAY-01	Webhook valid dari Midtrans	Pesanan PENDING, server key dikonfigurasi	POST /api/v1/payments/webhook {order_id, status_code, signature_key valid}	HTTP 200 OK, status_pesanan -> PAID	Status 200, status pesanan berubah ke PAID
PAY-02	Webhook signature tidak valid	Pesanan PENDING	POST /api/v1/payments/webhook {signature_key palsu}	HTTP 403 Forbidden	Status 403, status pesanan tetap PENDING
PAY-03	Webhook duplikat (idempotency)	Pesanan sudah berstatus PAID	POST /api/v1/payments/webhook {data valid, tapi pesanan sudah PAID}	HTTP 200 OK (tanpa duplikasi)	Status 200, tidak ada record pembayaran ganda

3.2.5 Spesifikasi Proses: Manajemen Logistik (Khusus Kurir)

Identifikasi>Nama: Modul Pelacakan Ekspedisi Fisik dan Validasi IDOR

Deskripsi Isi: Pembaruan titik koordinat fisik barang oleh Kurir sah . Jenis: RESTful API

Endpoint

- 3.2.5.1 Spesifikasi Tabel Input: Tabel pengiriman.
- 3.2.5.2 Spesifikasi Tabel Output: Tabel pengiriman (Update kolom status_kirim).

- 3.2.5.3 Spesifikasi Layar Utama (Endpoint API): Metode PATCH rute `/api/v1/shipping/:id_pengiriman/status`.
- 3.2.5.4 Spesifikasi Query: `UPDATE pengiriman SET status_kirim = ? WHERE id_pengiriman = ? AND id_kurir = ?;`
- 3.2.5.5 Spesifikasi Field Data Pada Layar (JSON Payload): `status_kirim` (Enum: DIKEMAS, DIKIRIM, DITERIMA).
- 3.2.5.6 Spesifikasi Function Key (Middleware): RoleMiddleware khusus "kurir".
- 3.2.5.7 Spesifikasi Layar Pesan (API Response): HTTP 200 OK.

Contoh Response Sukses Update Status (200 OK):

```
{
  "status": "success",
  "data": {
    "id_pengiriman": "uuid-xxx",
    "status_kirim": "DIKIRIM",
    "updated_at": "2026-03-08T14:00:00Z"
  }
}
```

Contoh Response IDOR Kurir Lain (403):

```
{
  "status": "error",
  "message": "Akses ditolak: Anda tidak ditugaskan pada pengiriman ini"
}
```

- 3.2.5.8 Spesifikasi Proses / Algoritma: 1) Validasi Role Kurir. 2) Cari manifest pengiriman. 3) Jika `id_kurir` di database tidak sama dengan pemohon, blokir dengan 403 Forbidden (Cegah IDOR antar kurir). 4) Jika sama, perbarui status.

State Chart modul Pengiriman: MENUNGGU KURIR (dibuat otomatis saat status pesanan = PAID) -> DIKEMAS (Admin tugaskan Kurir) -> DIKIRIM (Kurir update via PATCH endpoint) -> DITERIMA (Kurir konfirmasi diterima pembeli). Hanya Kurir dengan `id_kurir` yang cocok di manifest yang dapat melakukan transisi status (dilindungi validasi IDOR).

- 3.2.5.10 Spesifikasi Report (Matriks Pengujian SQA): Test Case SHP-01 (Pengujian Penetrasi Otorisasi Silang Kurir).

No TC	Skenario Uji	Precondition	Input / Aksi	Expected Result	Pass Criteria
SHP-01	Update status pengiriman sendiri (Happy Path)	Kurir login, <code>id_kurir</code> cocok di manifest	PATCH <code>/api/v1/shipping/uuid/status {"status_kirim":"DIKIRIM"}</code>	HTTP 200 OK + data pengiriman terbaru	Status 200, <code>status_kirim</code> berubah ke DIKIRIM
SHP-02	Update manifest milik Kurir lain (IDOR)	Kurir A login, manifest milik Kurir B	PATCH <code>/api/v1/shipping/uuid-kurir-b/status {"status_kirim":"DIKIRIM"}</code>	HTTP 403 Forbidden	Status 403, status manifest tidak berubah

SHP-03	Update dengan status tidak valid	Kurir login, manifest valid	PATCH /api/v1/shipping/uuid/status { "status_kirim": "HILANG" }	HTTP 400 Bad Request	Status 400, status tidak berubah
--------	----------------------------------	-----------------------------	---	----------------------	----------------------------------

3.3 Dekomposisi Fisik Modul

Bagian ini menjabarkan dekomposisi "fisik" dari struktur direktori kode sumber yang mengimplementasikan paradigma *Clean Architecture* pada bahasa pemrograman Golang. Dekomposisi ini memisahkan fungsionalitas sistem ke dalam modul-modul yang independen untuk mempermudah proses pemeliharaan (*maintenance*) dan pengujian unit (*Unit Testing*) .

Nama Direktori Fisik	Nama File	Nama Modul	Nama Fungsi	Keterangan Fisik & Logis
/internal/domain	user.go	Domain Layer	Pendefinisian Struct	Menyimpan entitas data murni dan <i>Interface</i> kontrak relasional tanpa logika operasional.
/internal/delivery/http	order_handler.go	Delivery Layer	CheckoutHandler()	Pengendali API; bertugas memvalidasi JSON masuk dan mengemas format respons HTTP.

/internal/usecase	order_usecase.go	Usecase Layer	ProcessCheckout()	Otak komputasi bisnis; mengeksekusi kalkulasi transaksi dan validasi ketersediaan stok aktual.
/internal/repository	product_repo.go	Repository Layer	UpdateStokTx()	Adaptor basis data; mengeksekusi sintaks kueri GORM menuju mesin PostgreSQL.
/pkg/jwt	jwt.go	Utility Library	GenerateToken()	Pustaka eksternal murni untuk enkripsi dan dekripsi <i>JSON Web Token</i> .
/cmd/api	main.go	Entry Point	main()	Titik mula <i>binary</i> peladen; membaca konfigurasi <i>environment</i> dan menyalakan

				<i>framework</i> Gin.

